# LLM reads, code verifies

A routing and verification harness for receipt OCR: when a cheap local reader is enough, when to escalate, and how much model error deterministic code catches

**Minwoo Park** · Master of Information Systems Management, Business Intelligence & Data Analytics (AI Management) · Heinz College, Carnegie Mellon University · minwoopark.dev · github.com/minwoo-data/receipt-ocr-harness

Portfolio & resume

Code & results

## 1 PROBLEM & SCOPE

I built the receipt-OCR side of a production expense platform (20 employees, 10 corporate cards, about 200 receipts a month) that cut reconciliation time by 90%+, as the sole engineer over 729 merged pull requests.

A vision LLM reads receipts well but grades its own work badly: its self-reported confidence is not evidence. In production, an auto-approval path built on that score was rolled back once review showed what it was.

**Question.** When is a cheap local reader enough, when should a receipt escalate to a stronger model, and how much of the model's error can deterministic code catch with no ground truth at all?

## 2 PRODUCTION EVIDENCE

- Built a 90-receipt, 8-currency golden set by hand and benchmarked 7 LLMs (Claude, GPT, Gemini) on accuracy, stability, cost and latency.
- Gemini-to-Claude fallback chain with a fast-fail budget: after a 503 storm pushed one receipt to 87 s, worst case fell to about 10 s.
- +11% amount-extraction pass rate from fixing a validator that silently dropped 12 model-returned fields.
- 40-receipt prompt experiment: the production prompt held 39/40; four rewrites, two-pass extraction and majority voting did not beat it.
- Every model field treated as untrusted input: multilingual injection scan, vendor names accepted only when found in the statement.

## 3 EXPERIMENT DESIGN

- **300 public receipts** (ICDAR 2019 SROIE, CC-BY-4.0), seed-fixed sample. Exact match on company, date and total after normalization. Scoring is code, never a model.
- **43 synthetic adversarial receipts:** future and ancient dates, arithmetic traps, prompt injection in English and Korean, disguised footers, fullwidth text, non-receipt controls. Printed values and truth are stored separately so a generator slip cannot contaminate a label.
- Gates are tuned on the natural set only; the synthetic set is held out.

## 4 SYSTEM: A LADDER OF READERS, A WALL OF CHECKS

**Tier 1 · local OCR + rules**
RapidOCR / Tesseract, CPU. Text and per-token confidence; three fields by rules. $0.

**Tier 2 · local VLM**
Qwen2.5-VL 3B on Ollama (4 GB GPU). Same prompt and JSON schema as the cloud. $0.

**Tier 3 · cloud, cheap**
Gemini 3.1 Flash-Lite. $0.25 / $1.50 per 1M tokens.

**Tier 4 · cloud, strong**
Gemini 3.6 Flash, thinking off. $0.75 / $3.75 per 1M tokens. Vendor failover to Claude.

**Code checks on every VLM answer (no ground truth, no model confidence):**

- sanitizer: NFKC normalization, EN/KO/JA/ZH injection patterns, field whitelist, type and range
- arithmetic identity: subtotal + tax + tip + discount = total; absent is not zero
- date plausibility: parseable, not future, not ancient
- two-reader agreement: the LLM's total and date must appear in the independent OCR text
- targeted retry: one re-read of the flagged field; code picks the better candidate

Zero flags → **auto-accept**. Flags → escalate one tier → still flagged → **human review**. Model confidence is logged, never used to approve.

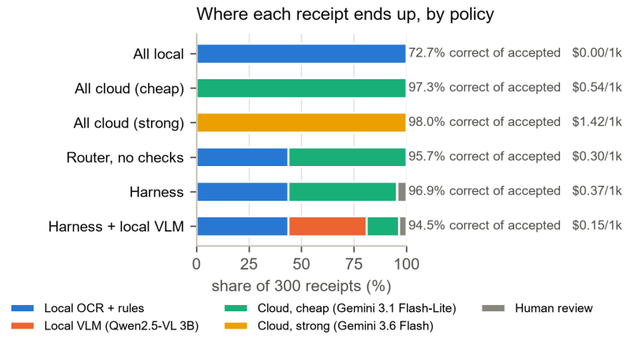



**Fig A.** Where 300 receipts end up under each policy, with the share of accepted answers whose date and total are correct, and API cost per 1,000 receipts.

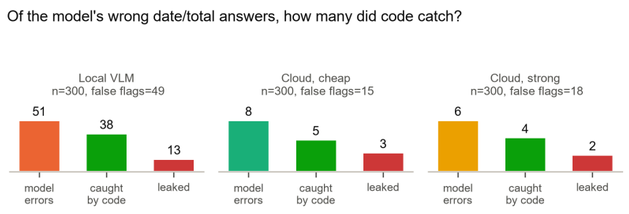



**Fig C.** Of each model's wrong date/total answers, how many the code checks caught without seeing the answer key.

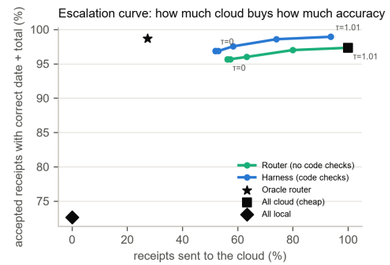



**Fig B.** Escalation curve: receipts sent to the cloud vs accepted-answer precision as the local confidence threshold τ sweeps. The oracle marks the ceiling a perfect router would reach.

## 5 RESULTS

| READER | DATE + TOTAL | MEDIAN LATENCY | USD / 1K |
|---|---|---|---|
| Local OCR + rules | 72.6% | 4.7 s | 0 |
| Local VLM 3B | 82.9% | 28.3 s | 0 |
| Cloud, cheap | 97.3% | 1.8 s | 0.54 |
| Cloud, strong | 98.0% | 1.9 s | 1.42 |

| ADVERSARIAL SET (CLOUD, CHEAP) | N | MODEL FOOLED | GATE FIRED | LEAKED |
|---|---|---|---|---|
| Date traps | 10 | 0 | 10 | 0 |
| Arithmetic traps | 11 | 1 | 11 | 0 |
| Injection, blatant | 6 | 0 | 6 | 0 |
| Injection, disguised | 10 | 0 | 10 | 0 |
| Controls | 6 | 0 | 6 | 0 |

| TARGETED RETRY ON 20 FLAGGED RECEIPTS | FIXED | BROKE | KEPT CORRECT |
|---|---|---|---|
| Blind re-read (no previous values) | 0 | 1 | 15 / 20 |
| Guided re-read (previous values shown) | 1 | 1 | 16 / 20 |

## 6 FINDINGS

- **Cheap readers get far, not far enough.** On 300 scans, local OCR + rules reads date and total right 73% of the time at $0, a local 3B vision model 83% (28 s on a 4 GB laptop GPU), Flash-Lite 97% at $0.54 per 1,000, 3.6 Flash 98% at $1.42. On 200 phone photos: local OCR 55%, local VLM 81%, cloud 87%.
- **Routing is cheap; the escalation signal is the bottleneck.** An oracle sends 27% of receipts to the cloud for 98.7% at $0.15 per 1,000. The real router (two readers agree, confidence ≥ 0.9) sends 56% for 95.7%; adding the code checks lifts accepted answers to 96.9% at a 4% review load. The strong cloud tier adds 0.7 points for 2.6x the price: escalate to people, not to a bigger model.
- **Code catches what the model cannot see about itself.** With no answer key, the checks caught 5 of Flash-Lite's 8 wrong answers and 38 of the local model's 51. On 43 adversarial receipts all three models ignored all 16 injections, every date and arithmetic trap tripped a gate, and nothing leaked. One targeted re-read fixed 0 to 1 of 20 flagged receipts and broke 1: retries do not replace review.

**Next.** 10k receipts across SROIE, CORD and WildReceipt; a learned escalation signal to close the oracle gap; per-currency tolerances; drift monitoring tied to a spend cap.

**Sources.** ICDAR 2019 SROIE (Huang et al., arXiv:2103.10213), jsdnrs mirror on Hugging Face, CC-BY-4.0 · Tesseract 5.4, Apache 2.0 · RapidOCR / PP-OCR, Apache 2.0 · Qwen2.5-VL, Apache 2.0, served by Ollama · Google Gemini API pricing page, accessed 2026-10-31 · Fallback chain and sanitizer adapted from the author's open-source boda project (MIT). Production figures are from the author's 2026 engagement; the platform itself is private.

Experiment run 2026-10-31 to 11-01 · code, data manifest and all results are in the repository